

University of Sheffield



COM3529 Software Testing and Analysis

Unit Testing

Dr José Miguel Rojas

Module Roadmap

Teaching Week	1	2	3	4	5	6	7	8	9	10
Topic	Course intro, theoretical foundations	Automation, testing at different levels	Unit testing, Avoiding brittle tests Good practice	Coverage criteria introduction	Coverage criteria: Control flow analysis, logic analysis	Regression testing	Model-based testing	Mutation testing	Metamorphic testing	Fuzzing and Search-Based Software Testing

The Nature of Unit Tests (Recap)

1. Narrow in scope
2. Limited to a single class or method
3. Small in size

Why Write Unit Tests?

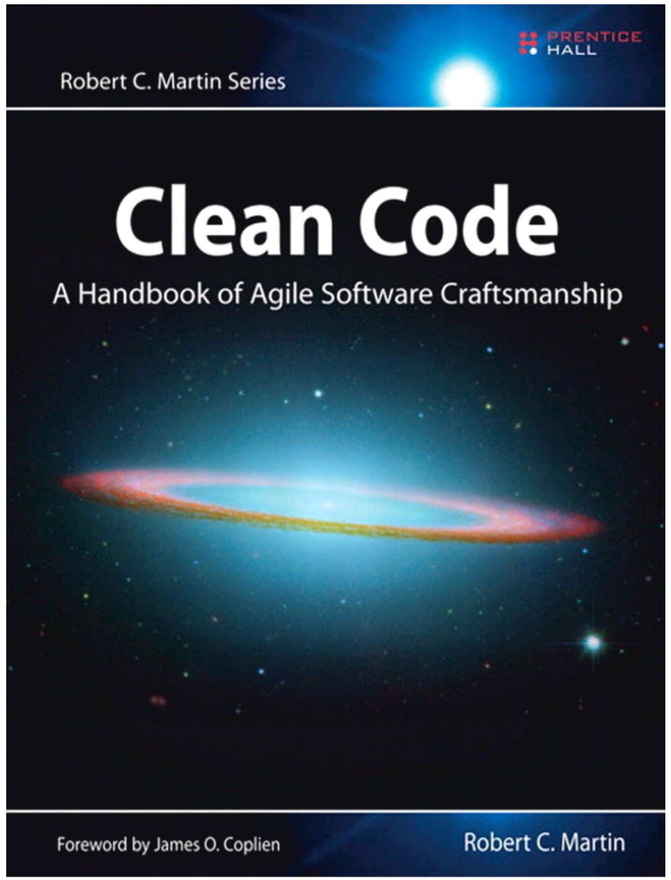
To prevent bugs (obviously!)

But also **to improve developer productivity** since unit tests:

1. **Help with implementation** – writing tests while coding gives quick feedback on the code being written.
2. **Should be easy to understand** when they fail – each test should be conceptually simple and focus on a particular part of the system.
3. **Serve as documentation** and examples of how to use the part of the system being tested (written documentation gets hopelessly out of date very quickly!).

At Google, 80% of tests are unit tests. The ease of writing tests and the speed of running them mean that engineers run several thousand unit tests a day.

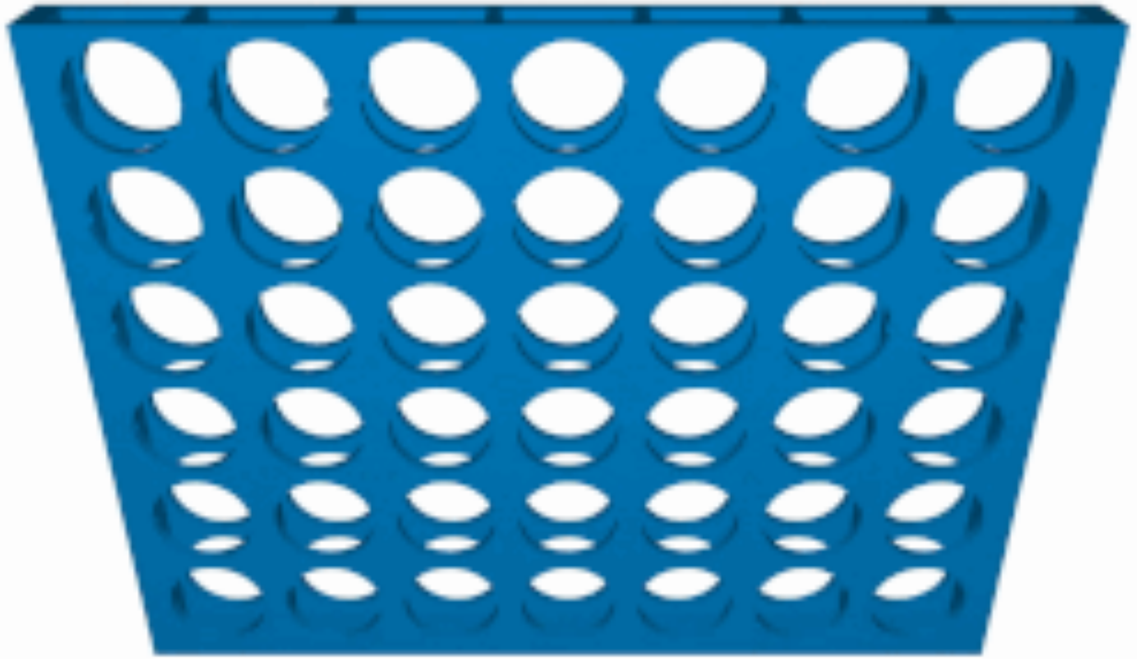
F.I.R.S.T. Principles



- **F**ast: Tests should run **quickly**, otherwise they can't be run *frequently*
- **I**ndependent: Tests should **not** depend on each other and execution order should **not** affect outcomes
- **R**epeatable: Tests should behave the same way in **any environment** (test, dev, personal laptop, etc)
- **S**elf-Validating: A test must have **a verdict** (output): either *pass* or *fail*
- **T**imely: Tests should be written alongside code, *or even before* (TDD)

How to *avoid* *Brittle* Unit Tests

The Connect4 Game



Silver Spoon, CC BY-SA 3.0, via Wikimedia Commons

To showcase examples of good and bad unit testing, we're going to look at tests written for class `Connect4` in package `uk.ac.shef.ac.uk.connect4` in the module's code repo.

An instance of the `Connect4` class represent the state of a game, including the positions of pieces in the grid, whose turn it is, etc.

The board is a 2D array of the `Piece` enum type

```
public enum Piece { RED, YELLOW; }
```

```
public class Connect4 {

    private static final int WINNING_SEQUENCE = 4;

    Piece[][] board;
    int cols, rows;
    Piece turn;
    boolean gameOver;
    Piece winner;

    public Connect4(int cols, int rows) {
        this.cols = cols;
        this.rows = rows;
        board = new Piece[cols][rows];
        turn = Piece.RED;
        gameOver = false;
        winner = null;
    }
```

“package private” members

public
methods

“package
private”
methods

```
    }

    public boolean isGameOver();

    public Piece winner();

    public Piece whoseTurn();

    public Piece getPieceAt(int col, int row);

    public void makeMove(int col);

    int firstAvailableRow(int col);

    boolean isValidCol(int col);

    boolean isValidRow(int row);

    boolean isBoardFull();

    boolean isGameWon();

    boolean isGameWon(int col, int row, int dCol, int dRow);
}
```

What's a Britle Test?

A unit test is brittle if it breaks in response to harmless and unrelated changes that introduced no real bugs.

The implementation of Connect4 needs to obey gravity. If a player drops a piece into a column, we need to ensure it ends up on the right row. The first piece will drop to the first row. The second piece in the same column will drop to the second row, etc.

We are going to test this functionality with two different tests.

> One tests the **implementation**, one tests **behaviour**.
>
> *Which one is more likely to have to change in future (i.e., be **brittle**)?*

Testing Implementation

Directly sets member variable

```
@Test
public void testFirstAvailableRow() {
    Connect4 c4 = new Connect4(7, 6);
    c4.board[0][3] = Piece.RED;
    assertThat(c4.firstAvailableRow(0), equalTo(4));
}
```

Verifies implementation using package-private method

What happens if we decide to re-implement the board differently?
(E.g., swap rows and columns in array, refactor the board out into a separate class entirely, etc.)

Testing Behaviour

```
@Test
public void shouldPlaceCounterAboveLast() {
    Connect4 c4 = new Connect4(7, 6);

    c4.makeMove(0); // RED
    assertThat(c4.getPieceAt(0, 0), equalTo(Piece.RED));

    c4.makeMove(0); // YELLOW
    assertThat(c4.getPieceAt(0, 1), equalTo(Piece.YELLOW));

    c4.makeMove(0); // RED
    assertThat(c4.getPieceAt(0, 2), equalTo(Piece.RED));

    c4.makeMove(0);
    assertThat(c4.getPieceAt(0, 3), equalTo(Piece.YELLOW));
}
```

This test is using the class's public API, to the extent of almost playing a game of Connect4.

The resulting behaviour (a change to the board) is then checked using a **public** method

Preventing Brittle Tests

Strive for **robust, unchanging tests** that **will not need to change** unless the project's requirements change by:

1. Test by calling **public methods only**.
2. Verify what results **are**, not **how** they are achieved.
3. Internal refactorings should **not** change the tests.
4. New features should leave existing ones **unaffected**.
5. Bug fixes **should not** require updates to tests.
6. **Behaviour changes** *may* require changes to tests.

Concentrating on testing implementation as opposed to behaviour tends to lead to brittle tests. So always prefer to test against behaviour.

How to Write *Clear* Unit Tests

Clear Unit Tests

- **Single concept** per test
  - Test a single concept/feature in each test (separation of concerns)
  - Use a **descriptive test name** that highlights the test's *intent*
- Write **effective assertions**
  - **Minimise** the number of **assertions** per test
  - A test must strive to fail for the same reason **and that reason only**
- **Split up tests** and use helper methods for common setups, e.g., using `@BeforeEach` in JUnit 5+

JUnit v. Hamcrest Assertions

The default supplied JUnit assertions have some deficiencies:

- **It's easy with the assertion method parameters to get expected and actual the wrong way round.** It's not terribly consequential, which is why it's easy to do, but the wrong ordering will confuse other programmers.
- **A different style is required for differing expected-actual relationships – i.e. a different assertion method.**
- The different assertion methods available are somewhat limited.
- It can be difficult to customise error messages.

The Hamcrest framework (hamcrest.org) provides us write more expressive, readable, and flexible assertions

Hamcrest Assertions

```
@Test
public void isocelesTest() {
    Triangle.Type result = Triangle.classify(5, 10, 10);
    assertThat(result, equalTo(Triangle.Type.ISOSCELES));
}
```

Every assertion uses the generic `assertThat` method

This assertion format better resembles natural language, making it more obvious that the first parameter is the **actual result** of the unit under test.

The relationship between actual and expected results is specified by a **matcher**, in this case, `equalTo`.

Hamcrest Matchers

Hamcrest has a plethora of “matchers”, like `equalTo`.

They can help write assertions involving a variety of types, including:

- Strings – e.g., can check if a string contains a substring, ignore case etc.
- Collections – whether an element is in a collection; what a collection contains, ignoring order etc.
- See **http://hamcrest.org/JavaHamcrest/javadoc/2.2/org/hamcrest/Matchers.html**

And if the appropriate matcher is not available it’s very easy to write your own.
See **https://www.baeldung.com/java-junit-hamcrest-guide**

Make Your Tests *Complete* and *Concise*

A test is complete when it contains all of the information a reader needs to understand how it arrives at its result.

```
@Test
// An incomplete test!
public void shouldEndGameWithWinnerWhenFourInARowVertically() {
    Connect4 c4 = new Connect4(7, 6);
    makeMoves(c4);
    assertThat(c4.isGameOver(), equalTo(true));
    assertThat(c4.winner(), equalTo(Piece.RED));
}
```

What's going on in this helper method?

Why **RED**? Where does that come from?

Make Your Tests *Complete* and *Concise*

A test is complete when it contains all of the information a reader needs to understand how it arrives at its result.

```
@Test
public void shouldEndGameWithWinnerWhenFourInARowVertically() {
    Connect4 c4 = new Connect4(7, 6);
    c4.makeMove(0); // RED
    c4.makeMove(1); // YELLOW
    c4.makeMove(0); // RED
    c4.makeMove(1); // YELLOW
    c4.makeMove(0); // RED
    c4.makeMove(1); // YELLOW
    c4.makeMove(0); // RED
    assertThat(c4.isGameOver(), equalTo(true));
    assertThat(c4.winner(), equalTo(Piece.RED));
}
```

Listing all moves makes the test longer and prevents re-using the move sequence as a helper method…

…***but***, it makes the test much easier to understand. Two columns of vertical pieces are being added to the board, and **RED** wins in column **0**

Don’t DRY Tests

DRY – Don’t Repeat Yourself: engineer needs to update one piece of code rather than tracking down all instances.

Downside: Can make code less clear, **requires following chains of references**. This might be a small price to pay for making the code easier to work with…

… but the cost/benefit analysis plays out differently with tests:

- **We want tests to break when software behaviour changes**
- **Production code has the benefit of a test suite to ensure it keeps working when things get complex. Tests should stand on their own!**
  (*Something has gone wrong when tests need tests*)

DAMP not DRY

DAMP: Descriptive And Meaningful Phrases

DAMP is not a *replacement* for DRY – it is complementary.

"Helper" methods can help make tests clearer by making them more **concise** – factoring out repetitive steps whose details aren't relevant to the behaviour being tested.

But the refactoring should be done with care for making the tests more readable and descriptive, not solely to reduce repetition.

Don't conflate clarity and conciseness.

Make Your Tests *Complete* and ***Concise***

A test is concise when it contains no other distracting or irrelevant information.

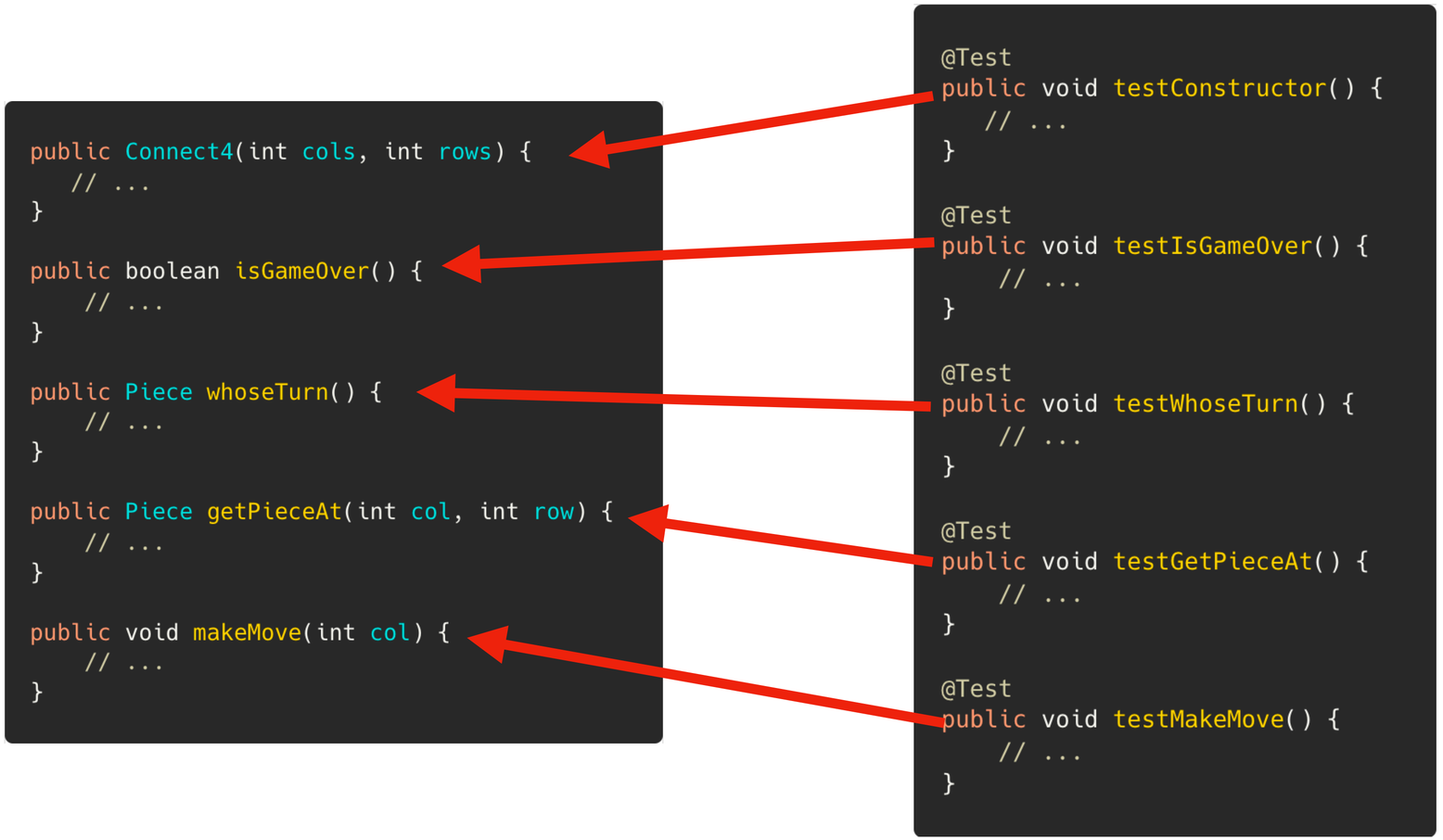
```
@Test
public void shouldEndGameWithWinnerWhenFourInARowVertically() {
    Connect4 c4 = new Connect4(7, 6);
    c4.makeMove(0); // RED
    c4.makeMove(1); // YELLOW
    c4.makeMove(2); // RED
    c4.makeMove(2); // YELLOW
    c4.makeMove(3); // RED
    c4.makeMove(3); // YELLOW
    c4.makeMove(0); // RED
    c4.makeMove(1); // YELLOW
    c4.makeMove(0); // RED
    c4.makeMove(1); // YELLOW
    c4.makeMove(0); // RED
    assertThat(c4.isGameOver(), equalTo(true));
    assertThat(c4.winner(), equalTo(Piece.RED));
}
```

Our test now includes several moves that are **not needed** for the test scenario.

It is now harder to visualise the board and to understand what the test outcome should be.

Don’t Test Methods – Test Behaviours

The first instinct of many software engineers is to try to match the structure of their tests to the structure of the code.



```
public Connect4(int cols, int rows) {
    // ...
}

public boolean isGameOver() {
    // ...
}

public Piece whoseTurn() {
    // ...
}

public Piece getPieceAt(int col, int row) {
    // ...
}

public void makeMove(int col) {
    // ...
}
```

```
@Test
public void testConstructor() {
    // ...
}

@Test
public void testIsGameOver() {
    // ...
}

@Test
public void testWhoseTurn() {
    // ...
}

@Test
public void testGetPieceAt() {
    // ...
}

@Test
public void testMakeMove() {
    // ...
}
```

Don’t Test Methods – Test Behaviours

But a single method may have more than one behaviour and/or some tricky corner cases that require more tests.

For example, `makeMove` can have several behaviours, depending on the state of the board.

One single test for that method makes no sense: it would likely not be **clear**, **complete** or **concise**.

More effective: Write tests for each **behaviour**!

Testing Behaviour

A behaviour is a guarantee that a system makes about how it will respond to a series of inputs while in a particular state.

A behaviour can be expressed with **“given X when Y, then Z”**

For example:

> **Given** a Connect4 board, with RED starting first
>
> **When** RED has played a piece
>
> **Then** it’s YELLOW’s turn next.

Writing Behaviour-Driven Tests

Behaviour-Driven tests tend to read more like natural language, so structure them accordingly.

```
@Test
public void shouldChangePieceAfterTurn() {
    // Given a Connect 4 Board, with RED starting first
    Connect4 c4 = new Connect4(7, 6);

    // When RED makes a move
    c4.makeMove(0);

    // Then it's YELLOW's turn next
    assertThat(c4.whoseTurn(), equalTo(Piece.YELLOW));
}
```

Name Tests after the Behaviour Being Tested

A good test name describes the actions (the “when”) that are being tested and the expected outcome (the “then”), and sometimes the state (the “given”).

A good strategy is to start the name with “should”, e.g.

```
shouldInitializeCorrectly
shouldChangePieceAfterTurn
shouldEndGameWithWinnerWhenFourInARowHorizontally
```
etc.

To complement this, since JUnit 5, a dedicated annotation exists:
`@DisplayName(“A more descriptive test name 😉”)`

Don’t Put Logic in Tests

Don’t put conditionals, logical operations or loops in tests.

Tests should read as simple statements of truth, *not chunks of code that also require tests!*

Here the test is trying to check every position of the board and ensure it is not set to a piece (i.e., it is null)

```
@Test
public void shouldInitializeCorrectly() {
    // Given a new Connect 4 Board
    Connect4 c4 = new Connect4(7, 6);

    // Then it's RED's turn
    assertThat(c4.whoseTurn(), equalTo(Piece.RED));

    // Then the board has no piece in every position
    for (int i=0; i < 7; i++) {
        for (int j=0; j < 6; j++) {
          assertThat(c4.getPieceAt(j, j), nullValue());
        }
    }

    // Then the game is not over
    assertThat(c4.isGameOver(), equalTo(false));

    // Then there is no winner
    assertThat(c4.winner, equalTo(null));
}
```

But oops, the developer made a mistake, checking `(j, j)` instead of `(i, j)`. The test won’t fail, and the mistake is hard to spot.

Don’t Put Logic in Tests

Don’t put conditionals, logical operations or loops in tests.

Tests should read as simple statements of truth, *not chunks of code that also require tests!*

Better to initialise a smaller 2x2 board and explicitly check each position

```
@Test
public void shouldInitializeCorrectly() {
    // Given a new Connect 4 Board
    Connect4 c4 = new Connect4(2, 2);

    // Then it's RED's turn
    assertThat(c4.whoseTurn(), equalTo(Piece.RED));

    // Then the board has no piece in every position
    assertThat(c4.getPieceAt(0, 0), nullValue());
    assertThat(c4.getPieceAt(0, 1), nullValue());
    assertThat(c4.getPieceAt(1, 0), nullValue());
    assertThat(c4.getPieceAt(1, 1), nullValue());

    // Then the game is not over
    assertThat(c4.isGameOver(), equalTo(false));

    // Then there is no winner
    assertThat(c4.winner, equalTo(null));
}
```

Making Your Unit Tests Clear

1. Make your tests **concise** and **complete** (DAMP and not too DRY!)
2. Structure tests **around behaviours** instead of around methods
3. Use the **Given-When-Then** pattern for testing behaviour
4. **Name Tests after the Behaviour Being Tested**
5. **Don’t put logic in tests**